



JavaScript:

The New Parts

ECMAScript2015 (ES6) is the specification that defines new features in JavaScript. In the second part of this series on JavaScript, let us look at new features like the binary/octal literals and enhanced object literals.

In the first part of this series, we covered the direction in which JavaScript (JS) is evolving. We also briefly looked at the new themes in the ECMAScript2015 specifications, before going on to cover block scope and template strings. And we explored two environments where you could try the new version—es6fiddle.net and [transpilrs](http://transpilrs.com).

In this part, we will cover more JS environments and look at the level of coverage in each of them. The browser environment has a ready to use debug environment.

Table 1 lists the level of coverage for features explained in the earlier article.

Table 1

(None: 0 per cent; Low: > 0-30 per cent; Medium: 30-70 per cent; High: 70-95 per cent; Full: 100 per cent)

Feature	Non-browser environments			Browsers (stable version)	
	io.js 3.0.0	Babel	node.js 0.12.0	Chrome 44	Firefox 40
Block scope	High	High	Low	High	Low
Template strings	Full	High	None	Full	Full

Language environments and their support

io.js: io.js is a famous fork of node.js that was created with the intention of having community control over the future of the compatible node.js environment. io.js has far more ES6 feature support than node.js. Learning new JavaScript features could be a compelling reason for you to move from node.js to io.js, which is a JS parser with a command line from which you can directly try ES6 features. Setting up io.js is covered in the next section.

Babel: This is more than a translator for ES6-to-ES5 conversion. Babel is recommended to JS enthusiasts who love

to understand the nuances of language better. You can see how ES6 features can be implemented with ES5. If you are keen to learn about compilers, you can do so by dissecting Babel, which has been developed by a JS enthusiast with the intention to learn JavaScript!

Node.js: Node.js support for ES6 features is limited. Please note that in some cases, you may need to enable a `--harmony*` flag to get the required feature support. For simplicity of representation, the need for flag-enabling is not explicitly mentioned in the table.

In browser environments, though the current version of Chrome seems to support ES6 better (w.r.t. features covered in Parts 1 and 2 of this series), the beta and nightly versions of Firefox have better overall ES6 support than Google Chrome. If you are a Google Chrome fan, consider switching to Firefox to experiment with ES6 from within browser environments.

Setting up io.js

Visit <https://iojs.org/en/index.html> and download the bundle. In the Linux (Ubuntu) environment, io.js is just a Zip file to be downloaded and unzipped. You can create a soft link to the node.js file, if you would like to have a single JS environment. io.js claims maximum compatibility with node.js. But if your intention to try io.js is only ES6, you can use io.js separately. Begin by issuing the following code:

```
janardan@ubuntu:~$ iojs --version
v3.1.0
janardan@ubuntu:~$ iojs
> greet = "Hello, io.js";
'Hello, io.js'
> console.log('Message: ${greet}');
Message: Hello, io.js
```